

Integration Technical Specification: PKCE-Backed API Handshake

This document defines the technical architecture, database designs, and sequence flow for establishing a secure, bidirectional, and multi-tenant integration between the in-house ERP System and the Public Job Platform.

1. Security Architecture Summary

- **Master Controller:** The ERP System acts as the primary orchestrator. It generates the api keys, handles tenant assignment, and manages the revocation lifecycle.
- **Proof Key for Code Exchange (PKCE):** Eliminates static shared `.env` secrets. Trust is established dynamically per-session.
- **Separation of Secrets (Key-Pair Architecture):**
 - **Key_Alpha (ERP Inbound):** Generated by ERP → Stored on Job Platform → Used by Job Platform to write to ERP.
 - **Key_Beta (ERP Outbound):** Generated by ERP → Stored on ERP → Used by ERP to write to Job Platform.
- **Storage Cryptography:** * Outbound keys are stored using reversible encryption (AES-256-GCM) so they can be parsed into HTTP headers.
 - Inbound validation keys are stored as **one-way SHA-256 hashes** to prevent system compromise if a database is breached.

2. Database Schema Design (PostgreSQL DDL)

A. Job Platform Database Schemas

The Job Platform houses the permanent connection records mapped to individual customer organizations, alongside a high-performance transient table for checking state parameters during user redirections. It also maintains a mapping table to link individual user accounts between both systems.

```
-- 1. Permanent Integration Mapping (Generic Partner Store)
CREATE TABLE connector_partner (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    organization_id VARCHAR(100) NOT NULL UNIQUE,
    partner_tenant_id VARCHAR(100) NOT NULL,           -- Stores the ERP's Company/Te
    partner_outbound_key TEXT NOT NULL,                -- Encrypted (AES-256-GCM) [Ke
    partner_inbound_key TEXT NOT NULL,                 -- Encrypted (AES-256-GCM) [Ke
    status VARCHAR(20) NOT NULL DEFAULT 'active',     -- 'active', 'disconnected'
    connected_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
```

```

CREATE INDEX idx_connector_partner_org ON connector_partner(organization_id);

-- 2. Transient Handshake / PKCE Store (Can also be managed via Redis with TTL)
CREATE TABLE connector_handshakes (
    temporary_code VARCHAR(64) PRIMARY KEY,          -- High-entropy unique key
    state VARCHAR(64) NOT NULL,                      -- Anti-CSRF token
    code_challenge VARCHAR(64) NOT NULL,             -- SHA-256 Hash of code_verifi
    organization_id VARCHAR(100) NOT NULL,
    expires_at TIMESTAMP WITH TIME ZONE NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Index for cleaning up expired handshakes
CREATE INDEX idx_connector_handshake_expiry ON connector_handshakes(expires_at);

-- 3. User Mapping between local accounts and Partner accounts
CREATE TABLE connector_user_mappings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    connection_id UUID NOT NULL REFERENCES connector_partner(id) ON DELETE CASCADE
    local_user_id VARCHAR(100) NOT NULL,             -- Unique User ID on the local
    partner_user_id VARCHAR(100) NOT NULL,           -- Corresponding User ID in th
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    -- Ensures a local user is only mapped to one partner account per active inte
    CONSTRAINT uq_connection_local_user UNIQUE (connection_id, local_user_id)
);

CREATE INDEX idx_connector_user_map_local ON connector_user_mappings(local_user_id)
CREATE INDEX idx_connector_user_map_partner ON connector_user_mappings(partner_user_id)

```

B. ERP System Database Schema

These tables belong inside each individual company's isolated database schema (if using database-per-schema multi-tenancy) or in the shared system schema with an index on `company_id`.

```

-- 1. Permanent Integration Mapping
CREATE TABLE connector_partner (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id VARCHAR(100) NOT NULL UNIQUE,          -- ERP Tenant / Company Identifier
    platform VARCHAR(50) NOT NULL DEFAULT 'connect_job', -- Target platform e.g.
    partner_outbound_key TEXT NOT NULL,              -- Encrypted (AES-256-GCM) [Key]
    partner_inbound_hash VARCHAR(64) NOT NULL UNIQUE, -- SHA-256 Hash of Key_Alpha
    partner_org_id VARCHAR(100) NULL,                -- Map reference to Job Platform
    status VARCHAR(20) NOT NULL DEFAULT 'active',    -- 'active', 'suspended', 'revoked'
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_platform CHECK (platform IN ('connect_job', 'another_system'))
);

```

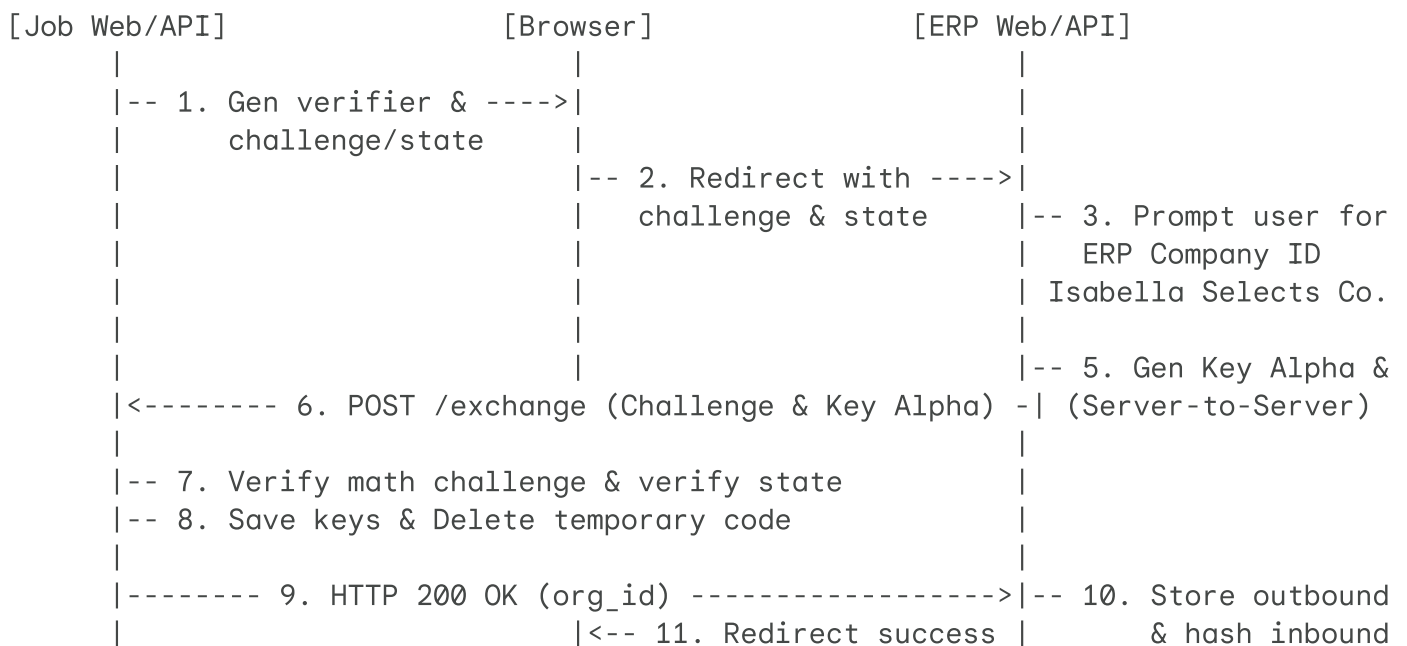
```
-- 2. User Mapping to link local ERP users to Job Platform users (for outbound sy
CREATE TABLE connector_user_mappings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  connection_id UUID NOT NULL REFERENCES connector_partner(id) ON DELETE CASCADE,
  local_user_id VARCHAR(100) NOT NULL,          -- Unique User ID in the ERP s
  partner_user_id VARCHAR(100) NOT NULL,        -- Corresponding User ID on th
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  -- Ensures an ERP user is only mapped to one partner account per active integ
  CONSTRAINT uq_connector_erp_local_user UNIQUE (connection_id, local_user_id)
);

CREATE INDEX idx_connector_user_map_erp_local ON connector_user_mappings(local_us
CREATE INDEX idx_connector_user_map_erp_partner ON connector_user_mappings(partne
```

3. Handshake Execution Flow

The sequence diagrams and API designs below describe the step-by-step transaction needed to link the platforms securely.

A. Sequence Diagram



4. REST API Endpoint Designs

Endpoint 1: The Token Exchange (Job Platform API)

- **URL:** POST <https://api.jobplatform.com/v1/integration/exchange>

JSON Request Body (From ERP Server):

```
{
  "temporary_code": "code_8ac93bfa09e3...",
  "state": "state_9103bc7e0a...",
  "erp_company_id": "erp_co_7718",
  "erp_outbound_key": "erp_key_alpha_secure_9238bc...",
  "code_challenge": "b579178129a3a938c823..."
}
```

Verification Steps (Job Platform Engine):

1. **Query:** Fetch handshake details from `connector_handshakes` where `temporary_code` matches request.
2. **Check Expiration:** Validate `expires_at > NOW()` . If expired, delete the row and throw `400 Bad Request` .
3. **Check CSRF State:** Assert that the incoming `state` matches the database `state` .
4. **Assert Challenge Matching:** Take the `code_verifier` stored securely in the user's browser-session cache. Run it through the SHA-256 generation function:

$$\text{Generated Challenge} = \text{Base64URL}(\text{SHA256}(\text{code_verifier}))$$

Validate that `Generated Challenge` is strictly equal to the `code_challenge` sent in the request body.

5. **Persist Keys:** If all assertions pass, commit keys to `connector_partner` and delete the matching record from `connector_handshakes` .

Success JSON Response Body (To ERP Server):

```
{
  "status": "success",
  "message": "Integration linked successfully",
  "job_platform_org_id": "job_org_9930"
}
```

5. Master Revocation (Disconnect) Flow

Because the ERP acts as the master authority, revocation deletes data downstream securely.

[ERP Web UI]	[ERP API Backend]	[Job Platform API]
-- 1. Click "Disconnect" ->		
	-- 2. Decrypt Key_Beta ----	
	-- 3. POST /disconnect ---->	(Auth Header: Bea
		-- 4. Lookup Key_
		-- 5. Delete Conn
	<-- 6. HTTP 200 OK -----	
	-- 7. Wipe ERP Database ----	
<-- 8. Show "Disconnected"		

Endpoint 2: Drop Integration (Job Platform API)

- **URL:** POST <https://api.jobplatform.com/v1/integration/disconnect>
- **Authentication:** Authorization: Bearer <Key_Beta>

Success Response:

```
{
  "status": "success",
  "message": "Connection terminated"
}
```

Upon receiving HTTP 200, the ERP backend immediately deletes its corresponding record from its